

C/C++方向面试题汇总

1. C/C++基础
2. Linux
 - 进程和线程
3. 网络
4. 数据结构与算法
 - 链表
 - Leetcode
 - 其它题目
5. 数据库
 - MySQL
6. 设计模式
7. 项目
 - 网盘项目
 - 搜索引擎项目
 - Http 项目
8. 开放性问题
 - 1. 项目相关
9. 面试真题
 - 真题1

C/C++方向面试题汇总

1. C/C++基础

1. 描述 C++ 程序的内存由哪几个部分组成，每个区域分别有什么作用和特点。
2. 什么时候分配内存会产生内存碎片？
3. 负数的编码方式是什么？简述一下它的原理。
4. 浮点数的编码方式是什么？简述一下它的原理。
5. 可执行程序是如何生成的？
6. 可执行程序是如何变成进程的？
7. 在C语言中如何调用C++函数？
8. 请描述几种常见的C/C++的缺陷和陷阱？
9. 重写，重载，重定义这三者有什么区别？
10. 说一说strcpy、sprintf与memcpy这三个函数的不同之处
11. strlen和sizeof区别
12. 二维数组是什么，函数指针是什么？
13. 简述值传递、指针传递（地址传递）的区别
14. C++中 `const` 关键字的作用？
15. C++中 `static` 关键字的作用？
16. C++中 `class` 和 `struct` 的区别？
17. string的 SSO 模式
18. C++中类型转换有哪几种？简述一下它们之间的区别。
19. C++中函数指针和指针函数的区别？

20. C++中有哪些智能指针，它们有什么异同？
21. void*的大小是多少？
22. 简述一下 malloc / free 的实现原理。为何 free(p) 的时候，只需要传递堆空间的地址就可以了？
23. malloc申请内存后，怎么保证一定申请到了呢？你会申请完后直接使用这片内存吗？
24. new / delete 与 malloc / free的异同
25. delete与delete[]
26. 指针和引用的区别
27. 引用作为函数返回时为什么不能返回局部变量？
28. extern"C"的用法
29. 内联函数和宏定义的区别
30. C++内存分为哪几块？有什么作用？
31. 静态变量什么时候初始化
32. 动态编译与静态编译
33. C++和C的struct区别
34. 拷贝构造函数的调用时机
35. inline函数的使用？缺点是什么？
36. 为什么拷贝构造函数必须传引用不能传值？
37. 类中静态函数占用内存么
38. 构造函数初始化和列表初始化的区别
39. 虚函数表的作用和存储的地址
40. 类指针初始化为空指针后调用成员函数会出问题吗？
41. 指针和引用的区别？
42. 动态指针的判空。
43. 构造函数可以设置成虚函数吗，为什么？
44. new和malloc之间有什么区别？new申请内存空间的时候为什么不需要类型转化，底层做了些什么事情呢？（new的作用:实例化一个对象。申请内存空间，然后调用相关的构造函数）
45. 虚函数表里存放的内容是什么时候写进去的？（编译阶段）
46. 虚函数和纯虚函数的区别
47. 为什么析构函数一般写成虚函数
48. 动态多态的实现过程，静态多态的实现过程
49. STL 中vector删除其中的元素，迭代器如何变化？为什么是两倍扩容？释放空间？
50. map、set是怎么实现的，红黑树是怎么能够同时实现这两种容器？为什么使用红黑树？
51. 模板和实现可不可以不写在一个文件里面？为什么？
52. 请简述你了解及使用过的C++11的新特性（提示：如 类型推导、智能指针、lambda匿名函数等）
53. 说一说你了解的关于lambda函数的全部知识
54. C++中的智能指针？三种指针解决的问题以及区别？
55. shared_ptr使用的注意事项？
56. weak_ptr如何解决shared_ptr的循环引用问题？
57. 假设有一个指针，如何做到多次使用，一次释放？

58. 分别写出在 if 语句的条件中，如何判断 bool a, int a, float a, char *a 是否为“零”。
59. 以int变量为对象，设计一个类，给出所有成员函数的声明(类似格式的声明只需要写一个)。
60. 请解释32位/64位系统具体指的是什么长度，对系统有何影响？
61. 简述系统物理内存与虚拟内存之间的联系与区别。
62. 简述你熟悉的编译器的不同的优化级别，以及编译器优化一些基本的思想。
63. 函数 `bool less(float x, float y) { return *(int *)&x < *(int *)&y; }` 是否能正确计算float的大小关系，为何？对应了c++的那种类型转换？
64. 有一个 `char*p={"abcdefg...z"}` 的字符串，这个字符串强转成int型指针，再进行自增运算，求该值。
65. 谈一下模板 template。
66. 定义函数时，缺省函数的返回值类型，则函数的返回值类型为：

- 1 A float
- 2 B int
- 3 C char
- 4 D void

67. 要实现动态连编，必须使用（）调用虚函数？

- 1 A 类名
- 2 B 派生类指针
- 3 C 对象名
- 4 D 基类指针

面向对象

68. 面向过程和面相对象的区别？
69. 面向对象的三大特性有哪些？
70. 什么是多态？多态分为哪几种，多态的应用场景有哪些？
71. 空类里有什么函数？
72. A继承B、C两个空类，对A进行强转成B、C，地址空间有什么变化呢？
73. public/private继承的关系
74. 公有继承和私有继承的应用场景？
75. C++的多态如何实现
76. 重载和覆盖的使用？
77. 基类和派生类的构造函数和析构函数的执行顺序？
78. 谈谈深拷贝和浅拷贝，以及如何实现？
79. string 的赋值操作是深拷贝还是浅拷贝？
80. 什么时候重载赋值运算符与复制拷贝函数？
81. 什么地方需要用到拷贝构造函数？
82. virtual()=0 是什么意思？
83. 虚函数和虚继承是怎么实现？
84. 如果我有一块地址空间，我怎么在这个地址空间内调用构造函数？

85. sizeof(A) 是多少?

```
1  class A {  
2      int a;  
3      short b;  
4      double c;  
5      virtual void fun() {}  
6      static int d;  
7  };
```

STL

86. STL包括哪些内容?

87. vector底层实现

88. vector 和 deque 的区别?

89. vector 和 list 的本质区别?

90. vector、list在添加删除的效率方面有什么不同?

91. 释放vector的内存的处理方式

92. vector迭代器失效的情况有哪些?

93. map和 unorderedmap的区别

94. stl 中的unordered_map和unordered_set有什么区别呢?

95. 自己实现unordered_map的话, 你会考虑到什么问题呢?

96. clear和erase的区别?

97. 迭代器失效问题。

98. swap函数的作用?

99. 简述一下STL容器相关知识、特性等?

100. stl 当中vector、list、map在内存中的数据结构有什么区别?

101. erase的返回值

2. Linux

1. 说说Linux中常用的命令?

2. 创建软连接的命令是什么?

3. /proc文件夹下放的是什么?

4. Linux下有哪些文件类型?

5. Linux查看内存、磁盘、端口、进程、线程命令有哪些?

6. gdb的基本使用

7. 是否在linux系统下用过gdb或者别的调试工具, 对于gdb来说, 用过哪些功能?

8. gdb 用法如果堆栈的信息不准, 怎么办 (不能运行, 不能修改代码), 可能是哪里出了问题

9. 如果某个模块运行过程中突然崩溃, 但是崩溃的几率不大, 如何定位并解决这个问题?

10. 如果是在一个循环内出现问题, 使用gdb调试需要等待很长时间, 应该怎么办?

11. 内存泄漏怎么检查, 怎么避免?

12. 什么是 coredump 文件? 怎么调试?

13. 什么时候用静态库和什么时候用动态库, 两者有何区别?

14. 零拷贝技术有哪些?

15. mmap的应用场景有哪些?

16. linux文件系统读入文件的过程。

进程和线程

4. 线程和进程区别？
5. 中断和异常的区别
6. 进程和线程的本质区别
7. 多进程和多线程的区别是什么？换句话说，什么时候该用多线程，什么时候该用多进程？
8. 进程间通信方式？
9. 线程间通信的方式？
10. Linux程序运行找不到动态库.so文件的三种解决办法
11. Linux进程同步的机制
12. 什么是同步、异步？什么是阻塞、非阻塞？
13. 进程间通信的方式有哪些？
14. 进程的六种状态
15. 什么是孤儿进程？什么是僵尸进程？怎么避免僵尸进程？
16. 结束进程的方式有哪些？
17. 什么是会话(session)？
18. 守护进程与后台进程的区别
19. 写时拷贝
20. 自旋锁
21. 谈一下对多线程的理解，如生产者 - 消费者问题。
22. 什么是死锁？死锁产生的条件？怎么解决死锁问题？
23. 信号量处理耗费多长时间，信号量同步会有什么问题
24. 登录 shell 进程是如何启动的？shell 是如何调用系统调用的？
25. sleep()调用后进程有哪些过程，在sleep()的过程中进程占用CPU了吗？
26. 线程池有什么好处？
27. 讲一下线程池？
28. 什么是线程安全？
29. 多线程间共享数据，用什么方式来保存它们的安全性
30. 什么是线程安全函数，工作中如何保证线程安全
31. 可重入函数是什么意思，为什么一定是线程安全的

3. 网络

1. 简述七层模型和四层模式。
2. 请描述一下从输入URL到显示页面的全过程。
3. 简述一下 socket 编程的流程。
4. write 阻塞的原因有哪些？
5. 多路复用：select, poll, epoll 的区别？epoll 的底层是如何实现的？
6. epoll边沿触发具体实现方式
7. LT和ET的区别，应用场景？
8. 说说同步、异步、阻塞、非阻塞。
9. 调用send函数发送数据不全怎么办？
10. 1G的文件从A机器发送到B机器，怎么发？(写代码实现)
11. 什么是 TCP 的"粘包"问题？怎么解决？
12. tcp和udp的区别？
13. tcp三次握手建立连接的过程？三次握手过程通信双方各自的状态？
14. tcp四次挥手的過程？四次挥手过程中通信双方各自的状态？
15. 简述一下 tcp 的超时机制。

16. tcp通信过程的状态是如何变化的
17. 从实用的角度来讲，三次握手的真实目的？（从硬件的角度来看，每一次握手的意义？）
18. 网络的七层模型？每一层的协议？
19. 为什么TIME_WAIT状态需要经过2MSL才能返回到CLOSE状态
20. TCP 利用滑动窗口实现流量控制的机制？
21. 如何根据IP获取对方的MAC地址？ ARP协议了解一下
22. Proactor和Reactor的区别和特点。
23. 怎样加快大文件在网络中传输，根据滑动窗口与拥塞控制考虑。
24. http和https的区别？
25. HTTP有哪些常用方法？ HTTP端口号？
26. SSH基于TCP还是UDP？ 端口号？
27. 讲一下WLAN？
28. 网卡的中断、几级缓存、网络方面遇到瓶颈怎么解决？
29. 什么时候会产生time_wait，如果系统出现大规模time_wait怎么处理？

4. 数据结构与算法

链表

1. 红黑树的思想，红黑树的特点，红黑树和平衡二叉树的区别？红黑树的查找为什么是 $O(\log n)$ ？
2. 负载均衡算法
3. 海量数据Top K问题
4. 有损压缩和无损压缩算法
5. 三个有序的序列，查找公共的部分
6. 合并两个有序数组或链表
7. 实现一个栈，实现 $O(1)$ 时间复杂度求栈的最小元素
8. 100G的文本，每行80k还是80字符，提示用多个机器，多进程，多线程，求出重复最多的行。一个机器内存8G，计算每个机器大概分多少？能读取100G的文本吗？找重复率前十的文本。
9. 两数之和(leetcode第一题)
10. 将奇数放于数组前面，偶数放于后面
11. 设计一个可以求最大值的栈
12. 余弦相似度算法
13. 如何判断单链表是否有环？
14. 反转单链表
15. 怎么判断单链表是否相交？
16. B树和B+树的特点以及应用场景？
17. hashmap实现的思路和方法(类似于STL中的unordered_map)
18. Hash表处理冲突的方式有什么？
19. 说一下什么是一致性哈希
20. 最短编辑距离问题

21. LRU 实现原理
22. 邻接矩阵是什么？
23. 贪心算法是什么，怎么保证最后的结果是最优的？

树

24. 二叉树的深度优先遍历：先序，中序，后序。
25. 二叉树层次遍历
26. 二叉树镜像反转
27. 假设数组为{3,5,7,4,6,2,1},构建一个二叉排序树,并输出排序结果
 - a)代码实现
 - b)简述思路

排序

28. 请实现冒泡排序
29. 请实现归并排序
30. 请实现快速排序
31. 快速排序算法思想？
32. 快排和堆排的应用场景。
33. 简述你所知道的 `std::sort` 实现方法。
34. 写一个排序函数，时间复杂度不大于 $O(n\log n)$ ，如果有多线程时该排序算法可如何优化，或者无法多线程优化的原因。

查找

35. 请设计一个目标字符串查询系统，输入一个目标字符串，找出磁盘上40亿个字符串中和目标字符串完全匹配的字符串，可以对40亿个字符串（数据量> 100GB，服务器可用内存为8GB）进行预处理，
 - a. 如果目标是**尽可能快**的完成字符串搜索工作，应该如何设计。
 - b. 如果不仅需要找到目标字符串还需要返回**按字典序**排在该字符串后面的字符串数量，又该如何设计呢？
36. 实现二分查找算法

Leetcode

8, 21, 39, 86, 127, 146

其它题目

1. 有一个包含4个字节的数 a，将它的每个字节相加，结果对 b 取模，如果结果小于c则称之为有效数，统计输入的数字中包含有效数的个数。
输入：c b 10个数 例：3 4 256 257 258 259 260 261 262 263 264 265
输出：有效数的个数 例：4
2. 如果一个字符串符合“辅音+元音(aeiou)+辅音(除了r)+e”的格式，则称它为有效字符串。输入一组字符串，用空格区分，如果单词中不包含非字母元素，则将单词翻转，统计翻转后的每个单词中包含有效字符串的个数，输出其中的最大值。
输入：一组字符串 例：!maxe a ekekac

输出：最大值 例：2

分析：!maxe a ekekac 翻转后为!maxe a cakeke。cakeke中有两个有效字符串cake和keke，故输出2。

3. solo 和 koko 是两兄弟，两人分一堆积木，每块积木都有自己的重量，弟弟koko要求积木的重量必须相等，否则就会哭（按照koko的标准），可是koko只会将数字转化为二进制后相加，而且总会忘记进位（每次都会忘记）。solo应该如何分积木，才能使自己得到的积木重量尽可能大，如果不能则输出-1？

输入：积木个数 每个积木的重量 例：3 3 5 6

输出：solo能拿到积木的最大重量 例：11

4. 给出字符串含 数字（0-9）和？ 例如（???1234?256??????3251?）其中？可以为任何数，求字符串转成数后 %13 = 5 的情况数量。

给的要求:字符串长度达到 10^5 ，数字前段可以为0

5. 已知某栅格三角网上三个顶点A(x,y,z),B(x,y,z),C(x,y,z),求三角网范围内任意点的高程z;
a)代码实现
b)简述思路
6. 实现全排列

5. 数据库

MySQL

1. 数据库事务是什么？
2. 数据库事务有哪些特性？
3. 事务的隔离级别有多少种，分别是什么？
4. 不可重复读和幻读区别是什么？可以举个例子吗
5. 什么是聚合索引？什么是非聚合索引
6. 数据库索引怎么用，适合什么场景，什么时候索引失效？
7. 如何对索引进行优化？
8. 创建索引一定能加快检索速度吗，为什么？
9. 为什么MySQL索引要使用B+树，而不是B树或者红黑树？
10. 你知道哪些数据库结构优化的手段
11. B树和B+树区别
12. 怎么判断一个查询是否是高效率的？
13. 如何优化查询语句？
14. MySQL的约束有哪些？
15. inner join, left join, right join, outer join 的区别？
16. mysql 如何合并两个表？
17. 共享锁与独占锁。
18. 乐观锁和悲观锁。
19. 了解过存储过程吗？
20. 了解过数据库视图吗？

Redis

21. Redis常见数据结构以及使用场景分别是什么？
22. Redis持久化机制可以说一说吗？
23. 了解Redis的线程模型吗？可以大致说说吗？
24. 有没有读过Redis源码
25. C++中的Map也是一种缓存型数据结构，为什么不用Map，而选择Redis做缓存？
26. Redis是如何部署的？
27. 简述一下主从复制和哨兵模式的原理？
28. 解释一下Redis缓存雪崩，缓存穿透，缓存预热？
29. redis的有序集合底层实现是什么？如果让你实现，你会怎么实现？
30. redis失效时应该怎么处理，如果让你设计你方案，你会怎么设计？

6. 设计模式

1. 为什么用组合而不要用继承？
2. 单例模式的构造函数？单例模式的创建过程？如何保证线程安全？
3. 手写单例模式
4. 如何使用单例模式，有什么注意事项？
5. 如果使用单例模式时创建了多个对象，如何定位问题？
6. 请简述一下适配器模式？
7. 实现一个简单的观察者模式
 - a)代码实现
 - b)简述思路
8. 使用过的设计模式，应用场景，如何应用？阐述业务背景和应用的的方式。

7. 项目

网盘项目

1. 客户端发消息给服务器，服务器端是如何解析的？
2. 多个用户上传同一份文件该如何处理？
3. 秒传如何实现？
4. 断点续传如何实现？
5. 讲一下虚拟文件目录
6. 前后端的通信方式知道哪几种？
7. 当你上传文件或更改文件时，如果出现问题，网络中断了；会不会导致数据库和文件对不上，怎么解决的？
8. 如何做token验证？
9. 网盘项目的网络通信方式
10. 有考虑过多线程同时上传一个文件的问题吗？
11. md5的算法是自己实现的吗？
12. 文件如何和用户绑定？
13. 连接使用的是长连接还是短链接？
14. 长连接socket的参数是怎么设置的？
15. 项目为什么要进行文件去重？
16. 线程池是如何实现的？

17. 线程池在项目当中是怎么用的，分别有哪些线程，它们是怎么分工的？
18. 线程池中的线程数目是否会随并发量动态增加？
19. 如何确定线程池中线程的状态？

搜索引擎项目

1. 一个网页的信息是通过什么形式存储的？
2. Simhash是什么，怎么使用的，用什么存储的？
3. 介绍一下倒排索引，余弦相似度算法。
4. 任务队列是怎么实现的，阻塞还是非阻塞？
5. 负载均衡怎么做的？
6. 什么是最短编辑距离，给你几个例子说说他们的最短编辑距离。
7. 双缓存轮换是怎么做到的，为什么不使用单缓存加锁？
8. LRU算法的原理。
9. 为什么使用LRU算法，还可以选用什么算法？
10. 如何加快搜索速度？
11. 缓存和数据库如何实现同步操作
12. 项目当中的任务队列具体会有哪些任务？
13. 搜索引擎项目用了几个进程几个线程池

Http 项目

1. 介绍一下这个workflow编程范式是什么，怎么用的，以及这个框架能对项目起到什么作用？
2. 有没有用什么二进制通信协议？

8. 开放性问题

1. 离职原因
2. 大学学过的和软件开发相关的课程？
3. 互联网以后的发展
4. 有没有看过开源代码和较深的数据结构？
5. 给一个尺子，算一栋楼的高度
6. 1000支试管，只有一支有毒，怎么用最少的小白鼠检测出有毒？
7. 最佩服的人
8. 最大的挫折
9. 有没有研究新技术
10. 毕业论文是什么？你对毕业论文的理解(这个论文的方向为什么吸引你)？
11. 你有什么想问的？
12. C++了解到什么程度？
13. 你之前工作小组怎么构成的？都干了多久？
14. 你之前公司是做什么业务的？
15. 之前工作累吗？工作时间之类的问题？
16. 你过去的工资怎么构成的？
17. 了解过我们公司吗？
18. 了解QT客户端开发吗？
19. 是否对OpenGL有了解？
20. 工作压力主要来自哪儿？
21. 工作过程中最难忘的一件事儿？
22. 未来的职业规划？
23. 进入公司后，一年的规划。
24. 项目需求怎么来的？
25. 项目怎么展开的？
26. 代码怎么测试的？

27. 测试用例自己写吗?
28. 多久做一个项目?
29. 修复bug的过程? 讲一个具体修复过的bug? 怎么发现的bug?
30. 如果某个模块运行过程中内存增长速度过快, 应该怎么处理?
31. 处理过的难度最高或挑战性最大的问题?
32. 有没有什么学习计划? 使用这种学习计划多长时间了, 会不会有完不成的情况?
33. 旁边有人不停打扰, 怎么办?
34. 你在找工作时更看中什么?
35. 接受加班么?
36. 期望薪资多少呢?
37. 对这个行业有什么看法
38. C++ primer中令你印象最深刻的是什么?
39. 是否会用UML?
40. 上一家公司的总体规模
41. 上一家公司是什么样的开发, 是敏捷式还是瀑布式还是别的类型的开发模式
42. 如何验证自己所写代码的质量?
43. 在以前的工作当中是否参与到某几款软件或者系统的设计工作/软件设计?

1. 项目相关

1. 介绍一下你的项目。
2. 整个项目做了多长时间? 项目多少个人?
3. 你的项目瓶颈在哪? 怎么优化的? 性能提升了多少?
4. 使用过什么测试性能的工具?
5. 项目中遇到的最大的问题是什么?
6. 你们系统的性能如何? 系统挂掉了怎么办? 系统挂掉后, 会不会有断流出现?
7. 项目组人数, 担任的角色, 工作的流程, 如何提交给测试?
8. 如果需要在已有项目上新增功能, 需要考虑哪些问题?
9. 项目有没有做过压测, 怎么做的?
10. 常用的云服务有用过吗? 它的原理是?
11. 私有云有没有用到集群

9. 面试真题

真题1

1. 每个C/C++头文件头部都需要使用类似如下的语句, 引入这种定义是为什么? (2分)

```
1 | #ifndef _C_CPP_BASIC_TEST_H
2 | #define _C_CPP_BASIC_TEST_H
3 |
4 | #endif
```

2. 当引用外部头文件时, 有以下两种引用包含方式:

```
1 | #include "myhead1.h"
2 | #include <myhead2.h>
```

- a) 预编译阶段, 编译器会对#include语句做如何的处理? (2分)
- b) 这两种引用方式的区别在于? (2分)
- c) 如果出现file not found的错误, 该如何解决? (3分)

3. 在C++中，通常需要使用命名空间，比如 `using namespace std;` 或者直接使用前缀指定 `std::function()`

a) C++为什么要使用命名空间？（2分）

b) 什么时候用using，什么时候用前缀的方式？（2分）

4. #define 的用法细节

a) #define 和 typedef 有什么区别？（2分）

b) 完成以下计算：

```
1 #define EQ(a) 0x12==a
2 EQ(12)=?      (2分)
3
4 #define ADD(a) a*2
5 ADD(1+1)=?    (2分)
6
7 #define TH(a) (!a||a==--2)
8 TH(1)=?       (1分)
9 TH(-2)=?      (1分)
```

c) 用 #define 来写宏函数，如上面的ADD(a)，有什么缺陷？该如何避免/改进（2分）

5. 字节计算（32位编译器）

```
1 struct A {
2     unsigned int a;//4
3     int b;//4
4 };
5
6 sizeof(struct A)=?    (2分)
```

```
1 struct B {
2     short a;//2
3     int b;//4
4 };
5
6 sizeof(struct B)=?    (2分)
```

```
1 const char* p = "string";
2
3 sizeof(p)=?          (2分)
```

6. 简述变量声明和定义的区别（3分）

7. 在C++文件中调用链接C语言代码函数，需要做什么特殊处理？（3分）

8. 以下代码编译时会报错，请找出报错原因（3分）

```

1  class A {
2  public:
3      A():data(0){};
4      ~A(){};
5
6      int data;
7      void inline inc(){data++;};
8      static void dec(){data--;};
9  };

```

9. 简述值传递、指针传递（地址传递）的区别（3分）

10. static 有多种不同的用法，比如加在变量上、函数上、成员函数上等等，这些用法本质上有什么共同之处？（3分）

11. 完成以下计算:

```

1  unsigned int sum = 0;
2  for (unsigned int i = 0; i < 10; i++) {
3      if (2 == ++i)
4          continue;
5      if (i == 5)
6          break;
7      if (i)
8          sum++;
9  }
10
11  跳出循环后sum=? （5分）

```

12. 为什么指针能指向函数，并能通过指针动态调用函数（多态），请简述其原理（3分）

13. 完成以下计算（32位编译器）

```

1  void func() {
2      unsigned char tmp[10] = {0};
3      tmp[0] = 0;
4      tmp[1] = 1;
5      tmp[2] = 2;
6      tmp[3] = 3;
7
8      unsigned char *p = tmp;
9      p++;
10     /*p=? （2分）
11
12     unsigned short *sp = (unsigned short *)tmp;
13     sp++;
14     /*sp=? （2分）
15 }

```

14. 在C++中引入模板，相比于C，其用处/优势在于？（2分）

15. 完成以下计算

```

1  u32 k = 7, m = 0;
2  switch (k) {
3  case 1:
4      m++;
5      break;
6  case 7:
7      m = 1;
8  default :
9      m = 0;
10 }
11
12 m=? (4分)

```

16. 举例三种常见的内存泄漏/溢出情景 (每个独立有效情景2分, 6分)

17. 完成以下计算 (32位编译器)

```

1  unsigned short a = 0, b;
2
3  void func(void)
4  {
5      /* 此时b=? */ (1分)
6      unsigned short a = 2;
7      a += 2;
8      /* a=? */ (1分)
9
10     b = 21;
11     b /= 2;
12     /* b=? */ (1分)
13
14     unsigned short c = 0x0001;
15     c = c << 8;
16     /* c=? */ (1分)
17
18     unsigned short d;
19     /* d的内存分配在哪里? 此时d的值为? */ (各1分)
20
21     short tmp = -100;
22     d = tmp;
23     /* 此时d的值为? */ (1分)
24
25     unsigned char e = 10;
26     e = e > 0 ? 1 : 0;
27     /* 此时e的值为? */ (1分)
28
29     u8 f = 0x01;
30     f &= 0x10;
31     /* 此时f的值为? */ (1分)
32 }

```

18. 简述C/C++编译过程, 必须涉及 .h, .c, .cpp, .o, .a 文件 (5分)

19. 英语测试二 (18分)

Design goals

There are myriads of JSON libraries out there, and each may even have its reason to exist. Our class had these design goals:

Intuitive syntax. In languages such as Python, JSON feels like a first class data type. We used all the operator magic of modern C++ to achieve the same feeling in your code. Check out the examples below and you'll know what I mean.

Trivial integration. Our whole code consists of a single header file json.hpp. That's it. No library, no subproject, no dependencies, no complex build system. The class is written in vanilla C++11. All in all, everything should require no adjustment of your compiler flags or project settings.

Serious testing. Our class is heavily unit-tested and covers 100% of the code, including all exceptional behavior. Furthermore, we checked with Valgrind and the Clang Sanitizers that there are no memory leaks. Google OSS-Fuzz additionally runs fuzz tests against all parsers 24/7, effectively executing billions of tests so far. To maintain high quality, the project is following the Core Infrastructure Initiative (CII) best practices.

Other aspects were not so important to us:

Memory efficiency. Each JSON object has an overhead of one pointer (the maximal size of a union) and one enumeration element (1 byte). The default generalization uses the following C++ data types: std::string for strings, int64_t, uint64_t or double for numbers, std::map for objects, std::vector for arrays, and bool for Booleans. However, you can template the generalized class basic_json to your needs.

Speed. There are certainly faster JSON libraries out there. However, if your goal is to speed up your development by adding JSON support with a single header, then this library is the way to go. If you know how to use a std::vector or std::map, you are already set.

请翻译此程序的任选三个功能特性，每个6分。